

CS480P/580P Introduction to Natural Language Processing

Patrick H. Chen

September 4, 2025

Lecture 5

Is this spam?

Dear Sir,

Please find attached the forms you requested.

Special Note: Student needs to provide the hand signed forms on the very first day of registration- (Forms attached)

IMPORTANT NOTE:

We will be needing additional bank statements and proof for justification if students have completed education outside their home state. (Legitimate documents- electricity bill, Rental Agreement, Bank Statement, College/University ID)

Please feel free to reach us in case any queries arise.

Study Abroad Partners

Positive or negative movie review?

-  unbelievably disappointing
-  Full of zany characters and richly applied satire, and some great plot twists
-  this is the greatest screwball comedy ever filmed
-  It was pathetic. The worst part about it was the boxing scenes.

Text Classification

Even language modeling can be viewed as classification!

- Let the set of classes be the words (vocabulary V)
- Predicting the next word is classifying
 - the context-so-far
 - into a class for each possible next word

Text Classification

Assigning subject categories, topics, or genres

Spam detection

Authorship identification (who wrote this?)

Language Identification (is this Portuguese?)

Sentiment analysis

Language modeling (what next word does this context expect)

Text Classification: definition

Input:

- a "document" (which can be any text) d
- a fixed set of classes $Y = \{y_1, y_2, \dots, y_J\}$

Output: a predicted class $\hat{y} \in Y$

The hat or circumflex notation $\hat{y}$ is used to refer to an estimated or predicted value

Text Classification

Most common classification method:
Supervised Machine Learning

Training

Input:

- a fixed set of classes $Y = \{y_1, y_2, \dots, y_J\}$
- a training set of m hand-labeled inputs $(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})$

Output:

- a learned classifier $y:d \rightarrow \hat{y}$

Inference or Test

Input: a document d

Output: a class $\hat{y}$

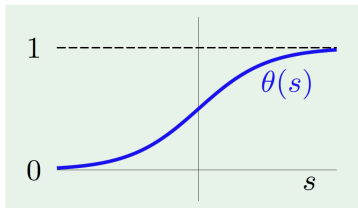
Logistic Regression

Logistic Regression

- Assume **linear** scoring function: $s = f(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$
- **Logistic hypothesis**:

$$P(y = 1 \mid \mathbf{x}) = \theta(\mathbf{w}^T \mathbf{x}),$$

$$\text{where } \theta(s) = \frac{e^s}{1+e^s} = \frac{1}{1+e^{-s}}$$

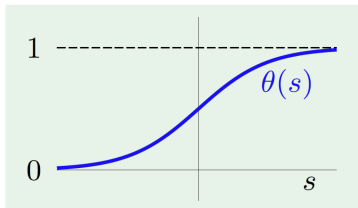


Logistic Regression

- Assume **linear** scoring function: $s = f(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$
- **Logistic hypothesis**:

$$P(y = 1 \mid \mathbf{x}) = \theta(\mathbf{w}^T \mathbf{x}),$$

$$\text{where } \theta(s) = \frac{e^s}{1+e^s} = \frac{1}{1+e^{-s}}$$



Logistic Regression

- Assume **linear** scoring function: $s = f(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$
- Logistic hypothesis:

$$P(y = 1 \mid \mathbf{x}) = \theta(\mathbf{w}^T \mathbf{x}),$$

$$\text{where } \theta(s) = \frac{e^s}{1+e^s} = \frac{1}{1+e^{-s}}$$

- How about $P(y = -1 \mid \mathbf{x})$?

$$P(y = -1 \mid \mathbf{x}) = 1 - \frac{1}{1+e^{-\mathbf{w}^T \mathbf{x}}} = \frac{1}{1+e^{\mathbf{w}^T \mathbf{x}}} = \theta(-\mathbf{w}^T \mathbf{x})$$

- Therefore, $P(y \mid \mathbf{x}) = \theta(y\mathbf{w}^T \mathbf{x})$

Maximum Likelihood Estimation

- Likelihood of $\mathcal{D} = (\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)$:

$$\prod_{n=1}^N P(y_n | \mathbf{x}_n) = \prod_{n=1}^N \theta(y_n \mathbf{w}^T \mathbf{x}_n)$$

- Find $\mathbf{w}$ to maximize the likelihood!

$$\begin{aligned} & \max_{\mathbf{w}} \prod_{n=1}^N \theta(y_n \mathbf{w}^T \mathbf{x}_n) \\ \Leftrightarrow & \max_{\mathbf{w}} \log(\prod_{n=1}^N \theta(y_n \mathbf{w}^T \mathbf{x}_n)) \\ \Leftrightarrow & \min_{\mathbf{w}} - \sum_{n=1}^N \log(\theta(y_n \mathbf{w}^T \mathbf{x}_n)) \\ \Leftrightarrow & \min_{\mathbf{w}} \sum_{n=1}^N \log(1 + e^{-y_n \mathbf{w}^T \mathbf{x}_n}) \end{aligned}$$

Empirical Risk Minimization

- Linear classification/regression:

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^N \text{loss}(\underbrace{\mathbf{w}^T \mathbf{x}_n}_{\hat{y}_n: \text{the predicted score}}, y_n)$$

- Linear regression: $\text{loss}(h(\mathbf{x}_n), y_n) = (\mathbf{w}^T \mathbf{x}_n - y_n)^2$
- Logistic regression: $\text{loss}(h(\mathbf{x}_n), y_n) = \log(1 + e^{-y_n \mathbf{w}^T \mathbf{x}_n})$

Empirical Risk Minimization

- Assume $f_{\mathbf{w}}(\mathbf{x})$ is the decision function to be learned
($\mathbf{w}$ is the parameters of the function)
- General empirical risk minimization:

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^N \text{loss}(f_{\mathbf{w}}(\mathbf{x}_n), y_n)$$

- Example: Neural network ($f_{\mathbf{w}}(\cdot)$ is the network)

Stochastic Gradient Descent

How to find the model?

- Logistic regression:

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^N \underbrace{\log(1 + e^{-y_n \mathbf{w}^T \mathbf{x}_n})}_{f_n(\mathbf{w})}$$

Now we have the Model, how do we find the $\mathbf{w}$?

How to find the model?

- Logistic regression:

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^N \underbrace{\log(1 + e^{-y_n \mathbf{w}^T \mathbf{x}_n})}_{f_n(\mathbf{w})}$$

Now we have the Model, how do we find the $\mathbf{w}$?

- Ans: Find the minimal point of the loss function.
- Linear Regression: Closed form solution.
- Logistic Regression: Gradient Based method.

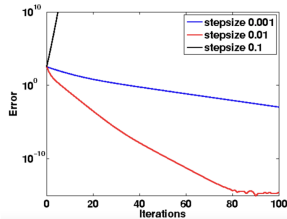
Gradient Descent

- Gradient descent: repeatedly do

$$\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t - \alpha \nabla f(\mathbf{w}^t)$$

$\alpha > 0$ is the **step size**

- Step size too large $\Rightarrow$ diverge; too small $\Rightarrow$ slow convergence



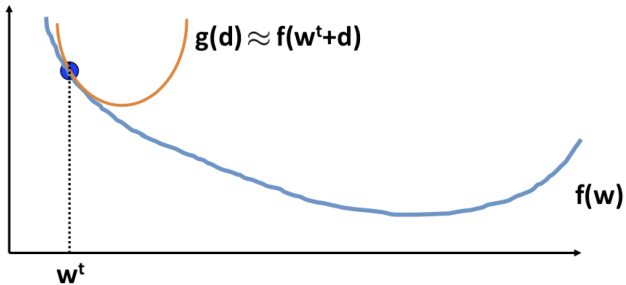
Why does it work?

- At each iteration, form an approximation function of $f(\cdot)$:

$$f(\mathbf{w} + \mathbf{d}) \approx g(\mathbf{d}) := f(\mathbf{w}^t) + \nabla f(\mathbf{w}^t)^T \mathbf{d} + \frac{1}{2\alpha} \|\mathbf{d}\|^2$$

- Update solution by $\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t + \mathbf{d}^*$
- $\mathbf{d}^* = \arg \min_{\mathbf{d}} g(\mathbf{d})$
 $\nabla g(\mathbf{d}^*) = 0 \Rightarrow \nabla f(\mathbf{w}^t) + \frac{1}{\alpha} \mathbf{d}^* = 0 \Rightarrow \mathbf{d}^* = -\alpha \nabla f(\mathbf{w}^t)$
- $\mathbf{d}^*$ will decrease $f(\cdot)$ if α (step size) is sufficiently small

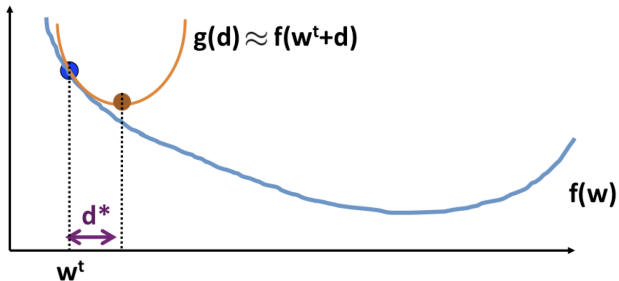
Illustration



Form a quadratic approximation

$$f(\mathbf{w}^t + \mathbf{d}) \approx g(\mathbf{d}) = f(\mathbf{w}^t) + \nabla f(\mathbf{w}^t)^T \mathbf{d} + \frac{1}{2\alpha} \|\mathbf{d}\|^2$$

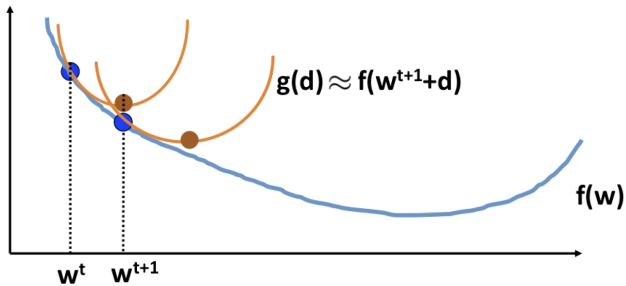
Illustration



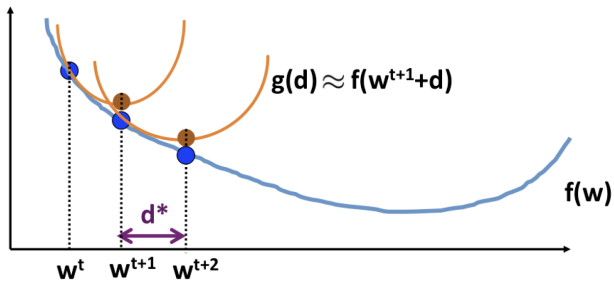
Minimize $g(\mathbf{d})$:

$$\nabla g(\mathbf{d}^*) = 0 \Rightarrow \nabla f(\mathbf{w}^t) + \frac{1}{\alpha} \mathbf{d}^* = 0 \Rightarrow \mathbf{d}^* = -\alpha \nabla f(\mathbf{w}^t)$$

Illustration



Illustration



Stochastic gradient

- Gradient:

$$\nabla f(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \nabla f_n(\mathbf{w})$$

- Each gradient computation needs to go through **all training samples**
slow when handling millions of samples
- Faster way to compute “approximate gradient”?

Stochastic gradient

- Gradient:

$$\nabla f(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \nabla f_n(\mathbf{w})$$

- Each gradient computation needs to go through **all training samples**

slow when handling millions of samples

- Faster way to compute “approximate gradient”?
- Use **stochastic sampling**:
 - Sample a small subset $B \subseteq \{1, \dots, N\}$
 - Estimated gradient

$$\nabla f(\mathbf{w}) \approx \frac{1}{|B|} \sum_{n \in B} \nabla f_n(\mathbf{w})$$

$|B|$: batch size

Stochastic gradient descent

Stochastic Gradient Descent (SGD)

- Input: training data $\{\mathbf{x}_n, y_n\}_{n=1}^N$
- Initialize $\mathbf{w}$ (zero or random)
- For $t = 1, 2, \dots$
 - Sample a **small batch** $B \subseteq \{1, \dots, N\}$
 - Update parameter

$$\mathbf{w} \leftarrow \mathbf{w} - \eta^t \frac{1}{|B|} \sum_{n \in B} \nabla f_n(\mathbf{w})$$

Extreme case: $|B| = 1 \Rightarrow$ **Sample one training data at a time**

Logistic Regression by SGD

- Logistic regression:

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{n=1}^N \underbrace{\log(1 + e^{-y_n \mathbf{w}^T \mathbf{x}_n})}_{f_n(\mathbf{w})}$$

SGD for Logistic Regression

- Input: training data $\{\mathbf{x}_n, y_n\}_{n=1}^N$
- Initialize $\mathbf{w}$ (zero or random)
- For $t = 1, 2, \dots$
 - Sample a batch $B \subseteq \{1, \dots, N\}$
 - Update parameter

$$\mathbf{w} \leftarrow \mathbf{w} - \eta^t \frac{1}{|B|} \sum_{n \in B} \underbrace{\frac{-y_n \mathbf{x}_n}{1 + e^{y_n \mathbf{w}^T \mathbf{x}_n}}}_{\nabla f_n(\mathbf{w})}$$

Why SGD works?

- Stochastic gradient is an **unbiased estimator** of full gradient:

$$\begin{aligned} E\left[\frac{1}{|B|} \sum_{n \in B} \nabla f_n(\mathbf{w})\right] &= \frac{1}{N} \sum_{n=1}^N \nabla f_n(\mathbf{w}) \\ &= \nabla f(\mathbf{w}) \end{aligned}$$

- Each iteration updated by

gradient + **zero-mean noise**